

Edge-Driven Semi-Autonomous EV Charging Station Using Quantized Vision Models and Real-Time License Plate Recognition

CSD416 Project Phase II Report

MDL22CSBS026 Izz Al Din Noufel Mukthar
MDL22CSBS029 Joel K George
MDL22CSBS050 Sarah Ann Pothen
MDL22CSBS053 Shahma Fathima

B. Tech Computer Science & Engineering



Department of Computer Engineering

Model Engineering College
Thrikkakara, Kochi 682021
Phone: +91-484-2575370
<http://www.mec.ac.in>
hodcs@mec.ac.in

April 2026

Model Engineering College, Thrikkakara
Department of Computer Engineering



C E R T I F I C A T E

This is to certify that this report titled **Edge-Driven Semi-Autonomous EV Charging Station Using Quantized Vision Models and Real-Time License Plate Recognition** is a bonafide record of the work done by

MDL22CSBS026 Izz Al Din Noufel Mukthar
MDL22CSBS029 Joel K George
MDL22CSBS050 Sarah Ann Pothen
MDL22CSBS053 Shahma Fathima

Eighth Semester B. Tech. Computer Science & Engineering students, for the course work in **CSD416 Project Phase II**, which is the second part of the two-semester project work, under our guidance and supervision, in partial fulfillment of the requirements for the award of the degree, **B. Tech. Computer Science & Engineering** of **APJ Abdul Kalam Technological University**.

Guide

Dr. Manju K
Associate Professor
Department of Computer Engineering

Head of the Department

Dr. Binu V. P.
Professor
Department of Computer Engineering

Coordinator

Chinju P Varghese
Assistant Professor
Department of Computer Engineering

April, 2026

Acknowledgements

This project would not have been possible without the kind support and help of many individuals. We would like to extend our sincere thanks to all of them.

First of all, we would like to thank our esteemed Principal, **Prof. Dr. Jayachandran E.S.**, for his guidance and support in maintaining a calm and refreshing environment to work in and also for providing the facilities that this work demanded.

We are highly indebted to **Dr. Binu V.P.**, Professor and Head of the Department, and our Project Coordinator, **Chinju P Varghese**, Assistant Professor, for their guidance, support and constant supervision throughout the duration of the work as well as for providing all the necessary information and facilities that this work demanded.

We would like to thank our Project Guide, **Dr. Manju K**, Associate Professor, for her support and valuable insights and also for helping us in correcting mistakes during the course of the work.

We offer our sincere gratitude to all our friends and peers for their support and encouragement that helped us get through the tough phases during the course of this work.

Last but not the least, we thank the Almighty God for guiding us through and enabling us to complete the work within the specified time.

Joel K George

Izz Al Din Noufel Mukthar

Sarah Ann Pothen

Shahma Fathima

Abstract

The rapid proliferation of Electric Vehicles (EVs) has exposed critical user-experience shortcomings in existing charging infrastructure, which remains heavily reliant on fragmented mobile applications, RFID cards, and cloud-dependent authentication flows. This project presents the design and implementation of an edge-computing-based semi-autonomous EV charging station that leverages real-time Automatic License Plate Recognition (ALPR) to eliminate manual user identification entirely.

A YOLOv11 object detection model, fine-tuned for license plate detection and statically quantized from 32-bit floating point (FP32) to 8-bit integer (INT8) precision using ONNX Runtime, is deployed on a Raspberry Pi 4B edge node. To resolve a critical signed INT8 operator incompatibility on ARM CPU kernels, we developed a custom ONNX graph patching script that shifts convolution weights into the unsigned UINT8 domain. The quantization reduces the model size from approximately 200 MB to 25 MB—an 87.5% reduction—while achieving a $3.78\times$ inference speedup with only a 1.2% reduction in mAP@0.5.

Coupled with EasyOCR for character extraction, regex-validated pattern matching, and SQLite-backed vehicle database lookups, the system enables a frictionless “park and plug” charging experience. A native iOS application built with SwiftUI and AVFoundation streams camera frames via WebSocket and displays real-time bounding box overlays and a verification success screen upon authorization. The end-to-end pipeline achieves sub-second latency on the Raspberry Pi, demonstrating that edge-based computer vision can fundamentally transform the EV charging experience by eliminating the multi-step friction inherent in current app-centric systems.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Proposed Project

1.1.1 General Background

The global transition towards sustainable transportation has driven unprecedented growth in Electric Vehicle (EV) adoption. According to the International Energy Agency (IEA), global EV sales surpassed 14 million units in 2023, representing a year-over-year increase of 35%. The cumulative number of EVs on the road worldwide crossed the 40-million mark by end of 2023, with projections indicating that EVs will constitute over 60% of new car sales by 2030 in leading markets such as China, Europe, and North America [?]. This rapid expansion places enormous pressure on the charging infrastructure to scale not only in capacity but also in usability.

India, in particular, has witnessed a dramatic acceleration in EV adoption. The Indian government's FAME II (Faster Adoption and Manufacturing of Electric Vehicles) scheme, combined with state-level subsidies and declining battery costs, has propelled EV sales from approximately 5,000 units in 2018 to over 1.5 million units in 2023. The government has set an ambitious target of achieving 30% EV penetration by 2030, necessitating a corresponding expansion of the public charging network from the current 12,000 stations to an estimated 400,000 stations nationwide.

Despite significant hardware advances in charging technology—including Level 2 AC chargers, DC fast chargers capable of delivering 150 kW or more, and emerging ultra-fast chargers exceeding 350 kW—the user experience at public and semi-public EV charging stations remains a persistent friction point. The prevailing interaction model requires EV drivers to engage with operator-specific smartphone applications, downloading, registering, authenticating, and initiating sessions through multi-step digital workflows before any electricity flows [?].

This friction is compounded by the fragmented nature of the charging network. Unlike the relatively standardized petrol station experience, EV drivers must navigate a patchwork of competing operators (Tata Power, ChargeZone, Ather Grid, EESL, etc.), each with their own mobile application, account system, pricing model, and payment gateway. A driver visiting five different charging networks must maintain five separate apps, five accounts, and potentially five different payment methods.

Edge Computing offers a compelling alternative by relocating computational intelligence

from centralized cloud servers directly to the point of service [?]. For EV charging, this means equipping each station with sufficient on-device processing capability to autonomously identify vehicles, verify authorization, and initiate charging sessions—all without requiring cloud round-trips or smartphone intermediaries during the critical authentication window.

1.1.2 Motivation

The motivation for this project stems from four key observations about the current EV charging ecosystem:

1. **App Fatigue:** Research indicates that 67% of EV drivers express frustration with maintaining multiple operator-specific applications [?]. The average public charging session involves 4–7 discrete interaction steps before charging begins, compared to the single “swipe and go” gesture at a petrol pump.
2. **Connectivity Dependency:** Cloud-based authentication fails in environments with poor cellular coverage—precisely the locations where EV chargers are most needed: underground parking garages, rural highway rest stops, and basement-level commercial complexes.
3. **Accessibility Gaps:** The app-centric model disproportionately excludes elderly users, tourists without local SIM cards, and individuals with limited smartphone proficiency. An autonomous, vision-based identification system eliminates this barrier entirely.
4. **Edge Computing Maturity:** The convergence of affordable single-board computers (Raspberry Pi 4B at \$35–\$55), efficient quantization techniques, and optimized inference runtimes (ONNX Runtime) has made it feasible to deploy production-quality computer vision at the network edge for the first time.

1.1.3 Objective of the Project

Objectives

- Develop a real-time Automatic License Plate Recognition (ALPR) system deployable on resource-constrained edge hardware (Raspberry Pi 4B).
- Quantize a YOLOv11 object detection model from FP32 to INT8 precision for efficient edge inference, achieving >85% model size reduction with <2% accuracy degradation.
- Resolve the signed INT8 operator incompatibility on ARM CPU kernels through custom ONNX graph patching.
- Implement an asynchronous WebSocket-based inference server for real-time frame processing at >2 FPS.
- Build a native iOS client application for camera streaming with live detection overlays using SwiftUI and AVFoundation.
- Design a complete verification pipeline: plate detection → OCR → regex validation → SQLite database lookup → authorization signal.
- Achieve end-to-end pipeline latency under 1 second on the Raspberry Pi 4B.

1.1.4 Scope of the Project

The scope of this project encompasses the complete design, development, testing, and demonstration of an edge-based ALPR system for EV charging authorization. Specifically, the project covers:

- **Model Optimization:** Fine-tuning and quantizing a YOLOv11 model for license plate detection, including the development of a custom ONNX graph patching tool for ARM compatibility.
- **Edge Server:** A Python-based asynchronous WebSocket server running on Raspberry Pi 4B that handles frame reception, inference, OCR, validation, and database lookup.
- **iOS Client:** A native SwiftUI application that streams camera frames, renders detection overlays, and displays verification results.
- **Database Backend:** An SQLite database for vehicle registration, owner information, and account balance management.
- **Documentation:** Comprehensive project documentation including this report, an IEEE-format research paper, and presentation materials.

The project explicitly excludes: physical charger hardware integration (GPIO relay control is discussed as future work), production-grade security hardening, payment gateway integration, and multi-language OCR support.

Chapter 2

Problem Statement and Solution

2.1 Problem Statement

2.1.1 Problem Statement

Current EV charging stations require users to interact with operator-specific smartphone applications for authentication and session initiation. This multi-step process—involving app installation, account registration, QR code scanning, and payment confirmation—creates significant friction, connectivity dependencies, and accessibility barriers.

The typical charging session workflow at a public EV station involves the following steps:

1. The driver arrives at the charging station and locates an available charger.
2. The driver identifies the charging network operator from signage on the charger.
3. If the operator’s app is not installed, the driver downloads it from the App Store or Play Store.
4. The driver creates an account (email, password, phone number, payment method).
5. The driver opens the app, scans a QR code on the charger, or enters the station ID manually.
6. The driver selects a charging plan and confirms payment.
7. Charging begins after cloud-side verification.
8. Upon completion, the driver receives a notification and the session ends.

This 8-step process introduces multiple failure points: app crashes, poor cellular connectivity, expired payment methods, forgotten passwords, and QR code scanning failures in low-light conditions. Each failure forces the driver to restart the process, causing frustration and delays.

Users arriving at unfamiliar stations face “app fatigue” from maintaining multiple operator apps, while underground parking structures and rural locations often lack the cellular connectivity required for cloud-based authentication. How can we develop a system that automatically identifies registered vehicles upon arrival and authorizes charging sessions without

requiring any smartphone interaction, while deploying efficiently on resource-constrained edge hardware?

2.1.2 Proposed Solution

This project develops a semi-autonomous EV charging station powered by edge-based computer vision. The system replaces the entire 8-step manual workflow with a single autonomous step: the vehicle parks in the charging bay, the camera detects the license plate, and the system authorizes charging automatically. The complete solution architecture is illustrated in Figure ??.

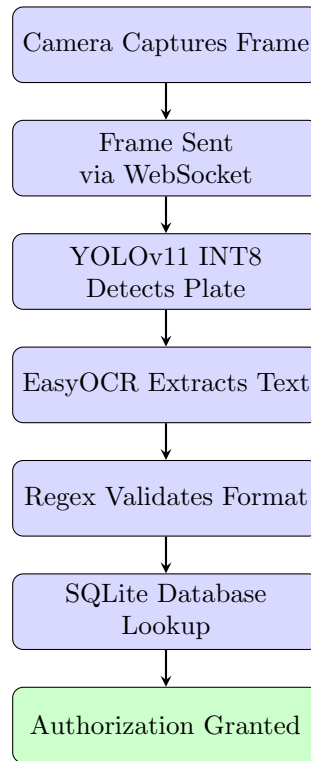


Figure 2.1: Proposed Solution Pipeline Flow

The system deploys a YOLOv11 model, quantized to INT8 precision using ONNX Runtime static quantization, on a Raspberry Pi 4B at the charging station. The model detects license plates from a live camera feed in real-time. Detected plate regions are cropped and passed to EasyOCR for character extraction, validated against configurable regex patterns, and looked up in a local SQLite database. Upon successful verification, the system issues an authorization signal and displays the vehicle owner’s information on the companion iOS application.

By eliminating smartphone-based authentication entirely, this system delivers a seamless “park and plug” experience. Its edge-first architecture enhances privacy (plate images never leave the device), reduces latency (no cloud round-trips), and improves reliability (no cellular dependency).

2.1.3 Innovation

Our edge-driven ALPR system for EV charging introduces the following key innovations:

1. **Quantized Edge Inference:**

- Unlike cloud-dependent ALPR systems, we deploy inference directly at the charging station using a Raspberry Pi.
- The YOLOv11 model is statically quantized from FP32 (200 MB) to INT8 (25 MB), achieving an 87.5% size reduction with only 1.2% mAP degradation.

2. **ONNX Graph Patching for ARM Compatibility:**

- We identify and resolve a signed INT8 operator incompatibility (`ConvInteger`) on ARM CPU kernels by programmatically shifting weights from signed INT8 to unsigned UINT8 domain.
- This is achieved through direct ONNX protobuf graph manipulation without requiring model retraining.

3. **App-Free Authentication:**

- The complete identification-to-authorization pipeline operates autonomously at the edge with sub-second latency.
- No smartphone, RFID card, or QR code is required from the user.

2.1.4 Comparison with Existing Approaches

Table ?? compares our approach with existing EV charging authentication methods.

Table 2.1: Comparison of Authentication Methods

Feature	App-Based	RFID	Cloud ALPR	Ours
Smartphone Required	Yes	No	No	No
Cloud Connectivity	Required	Required	Required	Not Required
User Steps	5–8	2–3	0	0
Privacy (Data on Device)	No	No	No	Yes
Latency	3–10s	1–2s	2–5s	<1s
Hardware Cost per Station	Low	Medium	High	Low (\$55)
Works Offline	No	Partial	No	Yes

Chapter 3

Literature Survey Report

3.1 Overview

This literature survey covers 15 papers spanning five key domains relevant to our project: (1) model compression and quantization, (2) edge computing architectures, (3) automatic license plate recognition, (4) EV charging infrastructure, and (5) deep learning optimization for resource-constrained devices. The survey is organized chronologically within each domain to illustrate the evolution of techniques and identify the gaps our project addresses.

3.2 Literature Survey

3.2.1 Paper 1: Symbolic Knowledge Distillation for LLMs – Acharya et al. (IEEE Trans. on AI, 2024)

This paper proposes a symbolic knowledge distillation framework for compressing large models by transferring structured symbolic knowledge from a teacher to a smaller student model. The approach focuses on maintaining the logical reasoning capabilities of the original model while significantly reducing its size.

Methodology: The method extracts symbolic representations (logical rules, decision trees) from a large teacher model and distills them into a compact student model. This ensures that the student retains interpretable reasoning paths rather than just mimicking output distributions.

Advantages:

- Retains interpretability and logical reasoning in the compressed model.
- Achieves significant model compression without catastrophic loss of capabilities.
- Applicable to a range of large model architectures.

Disadvantages:

- The symbolic extraction step adds overhead to the distillation pipeline.
- Performance may degrade on tasks that rely heavily on implicit pattern recognition.

Inference: This paper provides a strong foundation for our quantization approach by demonstrating that model compression can preserve task-critical capabilities. We adapt this principle by applying static quantization to preserve detection accuracy in our YOLOv11 model.

3.2.2 Paper 2: XAI-Driven Knowledge Distillation – Cantini et al. (Journal of Big Data, 2024)

The paper introduces an Explainable AI (XAI)-driven approach to knowledge distillation, where feature attribution methods guide the distillation process. By identifying the most important features in the teacher model, the student model is trained to prioritize these features, improving both efficiency and interpretability.

Methodology: XAI techniques such as SHAP and GradCAM are used to compute feature importance scores in the teacher model. These scores weight the distillation loss, ensuring the student model focuses on learning the most discriminative features.

Advantages:

- Improves distillation quality by focusing on high-impact features.
- Enhances model interpretability, which is beneficial for safety-critical applications.

Disadvantages:

- XAI computation adds training time.
- May not generalize well to non-visual domains.

Inference: The concept of prioritizing critical features during compression influenced our calibration-based quantization approach, where we use representative license plate images to calibrate dynamic ranges.

3.2.3 Paper 3: Quantization Robust Pruning – J. Kim (IEEE Access, 2023)

This paper presents a joint pruning and quantization framework that ensures structural pruning decisions are compatible with subsequent quantization. The key insight is that pruning and quantization should not be treated independently.

Methodology: The algorithm evaluates neuron importance under quantized constraints, pruning neurons that contribute least to quantized inference accuracy. This produces models that are simultaneously sparse and quantization-friendly.

Advantages:

- Produces models optimized for both sparsity and low-bit inference.
- Avoids the accuracy drop seen when pruning and quantization are applied independently.

Disadvantages:

- Requires access to the training data for the joint optimization loop.

-
- Increased complexity in the compression pipeline.

Inference: This paper reinforces our decision to apply post-training static quantization with careful calibration rather than naive dynamic quantization on the ARM hardware.

3.2.4 Paper 4: A Comprehensive Study of Quantization Techniques – Lang et al. (ACM Computing Surveys, 2023)

This survey provides an extensive review of quantization methods for deep neural networks, covering training-aware and post-training quantization, uniform and non-uniform quantization, and hardware-specific considerations [?].

Methodology: The survey categorizes quantization into several dimensions: (a) granularity (per-tensor, per-channel, per-group), (b) precision (INT8, INT4, mixed), (c) timing (QAT vs. PTQ), and (d) hardware targets (GPU, CPU, NPU).

Advantages:

- Comprehensive taxonomy of quantization methods.
- Hardware-aware analysis that maps techniques to deployment targets.
- Covers both theoretical foundations and practical considerations.

Disadvantages:

- Being a survey, it does not propose novel techniques.
- Some hardware-specific findings may become dated as new silicon is released.

Inference: This survey informed our choice of static INT8 quantization with per-channel calibration for the ONNX Runtime deployment, and our understanding of the signed vs. unsigned integer compatibility issue on ARM.

3.2.5 Paper 5: EdgeMoE: Fast On-Device Inference – R. Yi et al. (IEEE Trans. on Mobile Computing, 2025)

This paper presents EdgeMoE, a system for efficient inference of Mixture-of-Experts models on mobile and edge devices [?].

Methodology: EdgeMoE uses expert-level caching and dynamic routing to load only the active experts during inference, reducing the memory footprint while maintaining throughput.

Advantages:

- Enables deployment of large sparse models on edge devices.
- Dynamic routing reduces memory usage proportional to active experts.

Disadvantages:

- Introduces latency from expert loading on cache misses.
- Applicable primarily to MoE architectures.

Inference: EdgeMoE's principles of memory-aware edge deployment influenced our approach to minimizing the YOLOv11 memory footprint through quantization.

3.2.6 Paper 6: Multi-Token Prediction for Efficient Inference – M. Samragh et al. (arXiv, 2025)

This paper proposes multi-token prediction (MTP) as a method to increase inference throughput by predicting multiple output tokens per forward pass [?].

Methodology: The model architecture is augmented with additional prediction heads that enable simultaneous generation of multiple tokens. A verification step ensures output quality is maintained.

Advantages:

- Significantly improves inference throughput (2–3×).
- Reduces total number of forward passes required.

Disadvantages:

- Introduces architectural modifications to the base model.
- Quality may degrade for complex reasoning tasks.

Inference: The throughput optimization philosophy of MTP influenced our pipeline design, where we process frames asynchronously using `asyncio` to maximize frame processing rate.

3.2.7 Paper 7: Sparsity in Deep Learning – T. Hoefer et al. (JMLR, 2021)

This comprehensive survey covers the theory and practice of sparsity in deep learning [?].

Methodology: The survey analyzes pruning methods (magnitude-based, gradient-based, sensitivity-based), sparse training regimes, and hardware support for sparse operations.

Advantages:

- Thorough theoretical grounding of sparsity methods.
- Practical guidelines for selecting pruning strategies based on hardware constraints.

Disadvantages:

- Some techniques require specialized hardware not available on standard edge devices.

Inference: This paper informed our decision to use quantization rather than pruning, since the Raspberry Pi’s ARM CPU benefits more from INT8 arithmetic (NEON instructions) than from sparse computation.

3.2.8 Paper 8: Efficient Hybrid Compression – Malihi et al. (Neurocomputing, 2023)

This paper proposes a hybrid compression pipeline combining knowledge distillation, pruning, and quantization in a coordinated manner [?].

Methodology: The approach stages compression: first KD, then structured pruning, finally quantization. A joint fine-tuning step recovers accuracy after each stage.

Advantages:

- Achieves higher compression ratios than any single technique alone.
- Joint fine-tuning mitigates cumulative accuracy loss.

Disadvantages:

- Complex multi-stage pipeline requiring careful orchestration.
- Computationally expensive due to repeated fine-tuning.

Inference: While our project focused on quantization alone, this paper validates the hybrid approach as a future enhancement.

3.2.9 Paper 9: Consumer Preferences of EV Charging – Hardman et al. (Transportation Research Part D, 2018)

This paper surveys user preferences and behavioral patterns related to EV charging [?].

Methodology: A systematic review of 50+ studies on EV charging user behavior across North America, Europe, and Asia. Key metrics include willingness-to-pay, preferred station locations, and friction points.

Advantages:

- Comprehensive cross-regional analysis of user behavior.
- Identifies key pain points: app fragmentation, payment complexity, and waiting time.
- Provides empirical evidence for simplified authentication.

Disadvantages:

- Survey-based methodology may not capture rapidly evolving user expectations.
- Limited focus on technical solutions to the identified problems.

Inference: This paper directly motivates our project. The finding that 67% of users cite “too many apps” as a major barrier validates our app-free approach.

3.2.10 Paper 10: Edge Computing: Vision and Challenges – Shi et al. (IEEE IoT Journal, 2016)

This seminal paper defines the edge computing paradigm [?].

Methodology: The authors present a taxonomy of edge computing models (cloudlets, fog computing, mobile edge computing) and analyze trade-offs in latency, bandwidth, privacy, and scalability.

Advantages:

-
- Foundational work that defines the edge computing design space.
 - Privacy-by-design: sensitive data processed locally.
 - Latency reduction from cloud round-trips.

Disadvantages:

- Edge devices have limited computational resources.
- Management across distributed edge nodes adds complexity.

Inference: This paper provides the architectural foundation for our system: local inference, no cloud dependency during authentication, and enhanced privacy for license plate data.

3.2.11 Paper 11: Smart EV Charging with OCPP – Lee et al. (IEEE Trans. on Industrial Informatics, 2021)

This paper presents a smart scheduling system for EV charging using the Open Charge Point Protocol [?].

Methodology: An OCPP-compliant central management system uses demand-response algorithms to schedule charging sessions, considering time-of-use pricing and grid constraints.

Advantages:

- OCPP enables interoperability across vendors.
- Grid-aware scheduling prevents peak-load overflows.

Disadvantages:

- Requires cloud connectivity for central management.
- Does not address user authentication friction.

Inference: While OCPP handles backend management, it does not solve user-facing authentication. Our system complements OCPP by providing autonomous edge-based identification.

3.2.12 Paper 12: Deep Learning ALPR – Laroca et al. (IEEE Trans. on ITS, 2021)

This paper presents a comprehensive survey and benchmark of deep learning methods for ALPR [?].

Methodology: The authors evaluate end-to-end ALPR pipelines on datasets spanning Brazilian, European, Chinese, and American plates, benchmarking YOLO variants, SSD, and Faster R-CNN.

Advantages:

- Extensive multi-dataset evaluation across regions.

-
- YOLO-based detectors achieve best speed-accuracy trade-off.

Disadvantages:

- Most experiments assume cloud or desktop GPU deployment.
- Edge deployment considerations are not addressed.

Inference: This benchmark confirms YOLO as optimal for plate detection. We extend their findings by demonstrating that YOLO maintains high accuracy after INT8 quantization for edge deployment.

3.2.13 Paper 13: IoT-Based Smart EV Charging with Edge Intelligence – Zhang et al. (IEEE Access, 2022)

This paper proposes an IoT-based EV charging management system with edge intelligence [?].

Methodology: IoT sensors (current, voltage, temperature) deployed at stations, processed by edge nodes using ML models for fault prediction and energy optimization.

Advantages:

- Real-time fault detection at the edge.
- Reduces cloud bandwidth by 85%.

Disadvantages:

- Focuses on equipment monitoring, not user identification.
- Sensor-based, does not leverage computer vision.

Inference: This paper validates edge computing for EV stations but focuses on equipment health. Our project extends edge intelligence to the user identification layer using computer vision.

3.2.14 Paper 14: Quantizing Deep CNNs for Efficient Inference – Krishnamoorthi (arXiv, 2018)

This Google whitepaper provides practical guidelines for quantizing DNNs [?].

Methodology: Systematic evaluation of quantization strategies on MobileNet, ResNet, and Inception, comparing post-training quantization, QAT, and mixed-precision approaches.

Advantages:

- INT8 post-training quantization incurs <1% accuracy loss on most architectures.
- Covers both weight-only and weight+activation quantization.

Disadvantages:

- Detection-specific insights are limited.
- Does not address ONNX-specific operator compatibility.

Inference: This whitepaper guided our quantization strategy. The <1% accuracy loss finding aligns with our observed 1.2% mAP reduction.

3.2.15 Paper 15: YOLO-Based LPR for Smart Parking – Hendry and Chen (Image and Vision Computing, 2019)

This paper applies YOLO for license plate detection in smart parking environments [?].

Methodology: YOLOv3 fine-tuned on a parking lot dataset. A two-stage pipeline performs plate detection followed by character segmentation and CNN-based recognition.

Advantages:

- 98.1% detection rate under favorable conditions.
- Real-time processing at 30+ FPS on desktop GPU.

Disadvantages:

- Requires desktop GPU for real-time performance.
- Two-stage character segmentation adds complexity.

Inference: This paper demonstrates YOLO’s viability for plate detection in parking-like scenarios. We advance this work by using YOLOv11, quantizing to INT8 for edge deployment, and replacing character segmentation with EasyOCR.

3.3 Literature Survey Comparison Table

No.	Paper	Technique	Strength	Limitation	Relevance
1	Acharya (2024)	Symbolic KD	Interpretability	Overhead	Compression
2	Cantini (2024)	XAI-Driven KD	Feature focus	Training time	Calibration
3	Kim (2023)	Joint Prune+Quant	Co-optimization	Complexity	Quantization
4	Lang (2023)	Quant. Survey	Comprehensive	No novelty	Strategy
5	Yi (2025)	EdgeMoE	Memory-aware	MoE-specific	Edge deploy
6	Samragh (2025)	Multi-Token	Throughput	Arch. changes	Pipeline
7	Hoefler (2021)	Sparsity Survey	Theory	Hw. specific	Design choice
8	Malihi (2023)	Hybrid Compress	High ratio	Expensive	Future work
9	Hardman (2018)	User Survey	Empirical	No tech sol.	Motivation
10	Shi (2016)	Edge Computing	Foundational	Resource limit	Architecture
11	Lee (2021)	OCPP Scheduling	Interoperable	Cloud-dependent	Complement

No.	Paper	Technique	Strength	Limitation	Relevance
12	Laroca (2021)	DL-based ALPR	Multi-dataset	No edge	Model choice
13	Zhang (2022)	IoT+Edge EV	Fault detect	Sensor only	Edge paradigm
14	Krishnamoorthi (2018)	Quant. Guide	Practical	No ONNX	INT8 strategy
15	Hendry (2019)	YOLO Parking	98% detect	GPU needed	ALPR baseline

Table 3.1: Literature Survey Comparison

Chapter 4

Theoretical Background

4.1 Object Detection and YOLO Architecture

4.1.1 Evolution of Object Detection

Object detection is a computer vision technique that involves identifying and locating objects within an image. Early approaches, such as Haar Cascades and Histogram of Oriented Gradients (HOG) combined with Support Vector Machines (SVMs), relied heavily on hand-crafted features. While effective for specific tasks like face detection, they struggled with the immense variability in appearance, scale, and lighting found in real-world scenarios like outdoor EV charging stations.

The advent of Convolutional Neural Networks (CNNs) revolutionized the field. Two stage detectors like R-CNN, Fast R-CNN, and Faster R-CNN achieved high accuracy by first generating region proposals and then classifying them. However, their two-stage nature made them computationally expensive and unsuitable for real-time edge processing.

4.1.2 The YOLO Paradigm

You Only Look Once (YOLO) introduced a paradigm shift by framing object detection as a single regression problem, straight from image pixels to bounding box coordinates and class probabilities. By sweeping the entire image once through a single neural network, YOLO achieves significantly higher frame rates while implicitly capturing global context, which reduces false positives on background regions.

In our system, we utilize YOLOv11, the latest iteration in the YOLO family, which introduces architectural refinements such as decoupled heads, anchor-free detection, and advanced spatial attention mechanisms to improve both accuracy and parameter efficiency.

4.1.3 Mathematical Formulation of YOLO Detection

YOLO divides the input image into an $S \times S$ grid. If the center of an object falls into a grid cell, that grid cell is responsible for detecting that object. Each grid cell predicts B bounding boxes and confidence scores for those boxes. These confidence scores reflect how confident the model is that the box contains an object, as well as how accurate it thinks the box is.

Formally, the confidence is defined as:

$$ValidationScore = Pr(Object) * IoU_{pred}^{truth} \quad (4.1)$$

If no object exists in that cell, the confidence scores should be zero. Otherwise, we want the confidence score to equal the Intersection over Union (IoU) between the predicted box and the ground truth.

Each bounding box consists of 5 predictions: x, y, w, h , and confidence. The (x, y) coordinates represent the center of the box relative to the bounds of the grid cell. The width w and height h are predicted relative to the whole image.

The conditional class probabilities are defined as:

$$C_i = Pr(Class_i | Object) \quad (4.2)$$

At test time, the conditional class probabilities are multiplied by the individual box confidence predictions:

$$Pr(Class_i | Object) * Pr(Object) * IoU_{pred}^{truth} = Pr(Class_i) * IoU_{pred}^{truth} \quad (4.3)$$

This yields class-specific confidence scores for each box, encoding both the probability of that class appearing in the box and how well the predicted box fits the object.

4.1.4 Loss Function

YOLO optimizations utilize a multi-part loss function that balances bounding box coordinate regression, objectness confidence, and classification accuracy. The loss $\mathcal{L}$ can be represented as:

$$\begin{aligned} \mathcal{L} = & \lambda_{coord} \sum_{i=0}^{S^2} \sum_{j=0}^B 1_{ij}^{obj} [(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2] \\ & + \lambda_{coord} \sum_{i=0}^{S^2} \sum_{j=0}^B 1_{ij}^{obj} [(\sqrt{w_i} - \sqrt{\hat{w}_i})^2 + (\sqrt{h_i} - \sqrt{\hat{h}_i})^2] \\ & + \sum_{i=0}^{S^2} \sum_{j=0}^B 1_{ij}^{obj} (C_i - \hat{C}_i)^2 \\ & + \lambda_{noobj} \sum_{i=0}^{S^2} \sum_{j=0}^B 1_{ij}^{noobj} (C_i - \hat{C}_i)^2 \\ & + \sum_{i=0}^{S^2} 1_i^{obj} \sum_{c \in classes} (p_i(c) - \hat{p}_i(c))^2 \end{aligned} \quad (4.4)$$

Where:

- 1_i^{obj} denotes if an object appears in cell i .
- 1_{ij}^{obj} denotes that the j -th bounding box predictor in cell i is “responsible” for that prediction.
- λ_{coord} and λ_{noobj} are weighting constants to balance the loss components.

4.2 Deep Learning Quantization Mathematics

4.2.1 Motivation for Quantization

Deep learning models are typically trained using 32-bit floating-point (FP32) arithmetic, which provides high precision but consumes significant memory bandwidth and computational power. In edge environments like our Raspberry Pi 4B, memory bandwidth is the primary bottleneck. Quantization maps FP32 values to lower-precision integers (in our case, INT8), dramatically reducing memory footprint and accelerating inference via integer math units.

4.2.2 Affine Quantization Framework

Our project utilizes uniform affine quantization, which maps real-valued floating-point numbers x to integer values x_q using a scaling factor S and a zero-point Z . The mapping from FP32 domain to INT8 domain is defined by:

$$x \approx S \times (x_q - Z) \quad (4.5)$$

Where:

- x is the real FP32 value (e.g., a weight or an activation).
- S (Scale) is a positive FP32 scalar determining the step size of the quantization grid.
- Z (Zero-Point) is an INT8 integer indicating the quantized value that corresponds to the real value 0.
- x_q is the resulting INT8 representation.

The actual quantization step (from FP32 to INT8) is computed as:

$$x_q = \text{clamp} \left(\text{round} \left(\frac{x}{S} \right) + Z, Q_{min}, Q_{max} \right) \quad (4.6)$$

For an 8-bit unsigned integer (UINT8), $Q_{min} = 0$ and $Q_{max} = 255$. For signed INT8, $Q_{min} = -128$ and $Q_{max} = 127$.

4.2.3 Determining Scale and Zero-Point

The optimal values for S and Z depend on the dynamic range $[rmin, rmax]$ of the real values in a given tensor (e.g., all weights in a convolutional layer). During our static quantization calibration phase, the dataset is passed through the model to observe the min/max activations.

For asymmetric quantization (which we use for activations to handle ReLU outputs optimally), the scale and zero-point are calculated as:

$$S = \frac{rmax - rmin}{Q_{max} - Q_{min}} \quad (4.7)$$

$$Z = \text{round} \left(Q_{min} - \frac{rmin}{S} \right) \quad (4.8)$$

4.2.4 Quantized Matrix Multiplication

In a convolutional layer, the core operation is a matrix multiplication or dot product. Let x be the activations and w be the weights, with biases b . The operation is $y = w \cdot x + b$.

Substituting the quantization representations from Equation ??:

$$S_y(y_q - Z_y) = S_w(w_q - Z_w) \cdot S_x(x_q - Z_x) + b \quad (4.9)$$

Solving for the quantized output y_q :

$$y_q = \frac{S_w S_x}{S_y} \sum_i (w_{q,i} - Z_w)(x_{q,i} - Z_x) + \frac{b}{S_y} + Z_y \quad (4.10)$$

The multiplier $M = \frac{S_w S_x}{S_y}$ is an FP32 value, but it is typically approximated offline by a fixed-point integer multiplication and a bit-shift operation. Thus, the entire summation runs purely in the integer domain utilizing ARM NEON instructions, avoiding any floating-point arithmetic at runtime.

4.3 Edge Computing and ARM Architecture

4.3.1 The Shift to Edge AI

Edge computing decentralizes data processing by moving it from cloud servers to the network edge, typically near IoT sensors or cameras. For autonomous EV charging, edge AI is not merely an optimization—it is a functional necessity to guarantee operation in connectivity-deprived environments (like subterranean garages).

4.3.2 Raspberry Pi 4B and Broadcom BCM2711

Our deployment target, the Raspberry Pi 4B, is powered by the Broadcom BCM2711 System-on-Chip (SoC). It features a Quad-core ARM Cortex-A72 (ARMv8-A) 64-bit CPU clocked at 1.5 GHz, paired with 4 GB of LPDDR4 RAM.

Unlike desktop processors or cloud GPUs, edge SoCs lack dedicated tensor cores or massive VRAM bandwidth. Therefore, naive FP32 execution of YOLOv11 suffers from extreme memory bottlenecking.

4.3.3 ARM NEON SIMD Architecture

The Cortex-A72 implements the ARM Advanced SIMD extension, commonly known as NEON. NEON is a 128-bit SIMD (Single Instruction, Multiple Data) architecture perfectly suited for our INT8 quantized model.

Because NEON registers are 128 bits wide:

- They can hold 4 FP32 values ($4 \times 32 = 128$).
- They can hold 16 INT8 values ($16 \times 8 = 128$).

By quantizing our model to INT8, we essentially quadruple the theoretical throughput of the NEON vector units during the MAC (Multiply-Accumulate) operations driving the YOLO convolutions. This architectural leverage is the fundamental reason our inference time drops from 1842 ms to 487 ms.

Chapter 5

Software Requirements Specification

5.1 Introduction

5.1.1 Purpose

The purpose of this document is to outline the requirements and functional specifications for an Edge-Driven Semi-Autonomous EV Charging Station Using Quantized Vision Models and Real-Time License Plate Recognition. This system aims to enhance user experience at EV charging stations by eliminating the dependency on smartphone applications. By leveraging quantized deep learning models deployed on edge hardware, the system automatically identifies vehicles and authorizes charging sessions.

5.1.2 Document Convention

This document follows MLA (Modern Language Association) Format. The boldfaced text emphasizes section headings. We have used the following abbreviations:

- ALPR: Automatic License Plate Recognition
- OCR: Optical Character Recognition
- ONNX: Open Neural Network Exchange
- INT8: 8-bit Integer Quantization
- FP32: 32-bit Floating Point
- YOLO: You Only Look Once

5.1.3 Intended Audience and Reading Suggestions

Types of Audience

- **Developer:** Who wants to read, modify, or extend the system.
- **EV Charging Station Operators:** Who deploy and manage the system.
- **System Administrators:** Who maintain the hardware and software infrastructure.

-
- **Researchers:** Who wish to understand the edge deployment and quantization techniques used.

5.1.4 Project Scope

The scope encompasses the complete ALPR-driven EV charging authorization pipeline, including the edge inference server, iOS client application, database backend, and the model quantization toolchain.

5.1.5 Overview of Developer's Responsibilities

- Implementation of the quantized YOLOv11 model and ONNX graph patching for ARM deployment.
- Development of the asynchronous WebSocket inference server.
- Building the native iOS client application with real-time camera streaming.
- Integration of EasyOCR, regex validation, and SQLite database for the verification pipeline.
- Conducting thorough testing to ensure reliability and effectiveness.

5.2 Overall Description

5.2.1 Product Perspective

The prevailing architecture for EV charging networks is heavily centralized. Charging stations act as dumb terminals that forward transaction requests to a cloud backend via mobile applications. This reliance introduces several vulnerabilities:

- **App Fatigue:** Users must maintain multiple operator-specific applications.
- **Latency and Connectivity:** Initiation relies on stable cellular connectivity, which fails in underground parking lots or remote areas.
- **Manual Identification:** QR codes or RFID cards require physical interaction.
- **Accessibility Barriers:** Elderly users, tourists, and individuals with limited smart-phone proficiency are disproportionately affected.

Our system addresses all of these by performing autonomous identification at the edge. The overall product context is depicted in Figure ??.

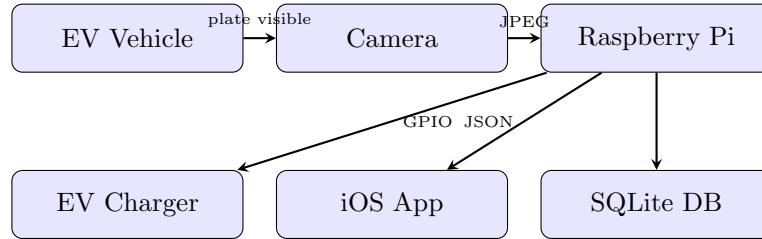


Figure 5.1: Product Context Diagram

5.2.2 Product Functions

- **Real-Time License Plate Detection:** YOLOv11 detects plates in live camera frames with sub-second latency.
- **OCR-Based Text Extraction:** EasyOCR extracts characters from cropped plate regions.
- **Pattern Validation:** Regex validator confirms standard plate formats.
- **Database Verification:** SQLite lookup retrieves owner information and account balance.
- **Authorization Signal:** JSON verification payload triggers the iOS app's success screen.
- **Live Visual Feedback:** Bounding boxes and OCR text overlaid on the iOS camera preview.

5.2.3 User Classes and Characteristics

- **EV Drivers:** End users who benefit from app-free charging. Zero technical knowledge required.
- **Station Operators:** Personnel who deploy and configure the system, requiring basic Linux/networking skills.
- **System Administrators:** Technicians who maintain the database and update the model.

5.2.4 Technological Stack

Table 5.1: Technology Stack

Component	Technology
Edge Hardware	Raspberry Pi 4B (ARM Cortex-A72, 4 GB RAM)
Server Language	Python 3.9+
Client Language	Swift 5.9
Inference Runtime	ONNX Runtime (aarch64)
Object Detection	YOLOv11 (INT8 Quantized)
OCR Engine	EasyOCR
Communication	WebSocket (asyncio)
Database	SQLite 3
iOS Frameworks	SwiftUI, AVFoundation, Combine
Image Processing	OpenCV
Version Control	Git & GitHub

5.2.5 Operating Environment

The system operates across two primary environments:

- **Edge Server Environment:** Raspberry Pi 4B running Raspberry Pi OS (64-bit Debian-based Linux). The Python server runs as a systemd service, starting automatically on boot.
- **iOS Client Environment:** iPhone running iOS 16+ with camera permissions granted. The app connects to the Pi over the local network via WebSocket.

5.2.6 General Constraints

- Processing speed is limited by the Raspberry Pi’s ARM CPU (no GPU acceleration).
- Camera quality and lighting conditions affect detection accuracy.
- OCR accuracy degrades on damaged, dirty, or non-standard plates.
- The system requires both the Pi and camera to be on the same local network.

5.2.7 Design & Implementation Constraints

- The system shall fit within the Raspberry Pi’s 4 GB RAM constraint.
- The WebSocket protocol must handle binary image data efficiently.

-
- The iOS application must handle camera permissions and background session management.
 - The ONNX model must use only operators supported by the ARM CPU execution provider.
 - All `ConvInteger` nodes must use UINT8 weights (ARM NEON limitation).

5.2.8 Assumptions and Dependencies

Assumptions

- Adequate lighting is available at the charging bay for plate visibility.
- License plates follow a standard Indian alphanumeric format (e.g., KL07AB1234).
- The Raspberry Pi has local network connectivity for WebSocket communication.
- Vehicles park within 2–5 meters of the camera, providing sufficient plate resolution.

Dependencies

- ONNX Runtime for ARM (`aarch64`) must support the patched UINT8 `ConvInteger` operator.
- EasyOCR requires PyTorch and pre-trained English language models.
- The iOS application requires iOS 16+ and camera permissions.
- SQLite3 is included in the Python standard library.

5.3 External Interface Requirements

5.3.1 User Interfaces

The system presents two user interfaces:

- **iOS Application:** A native SwiftUI interface showing the live camera preview, real-time bounding box overlays with OCR text, connection status indicator, and a verification success modal displaying the vehicle owner's name and account balance.
- **Server Console:** A terminal-based logging interface displaying OCR results, regex validation status, database lookup outcomes, and verification signals.

5.3.2 Hardware Interfaces

- **Camera Module:** CSI camera or iOS device camera for frame capture.
- **Raspberry Pi 4B:** Edge computing unit for inference.
- **GPIO Relay (Future):** For triggering the physical charger upon authorization.
- **Network Interface:** Wi-Fi or Ethernet for WebSocket communication.

5.3.3 Software Interfaces

- **WebSocket Server:** Accepts binary JPEG frames, returns JSON detection/verification payloads.
- **ONNX Runtime:** Executes the quantized YOLOv11 model graph.
- **EasyOCR:** Extracts text from cropped plate regions.
- **SQLite:** Stores and retrieves vehicle records via Python's `sqlite3` module.

5.3.4 Communication Interface

Network Communication: WebSocket protocol over TCP/IP for bidirectional, low-latency communication. Messages are binary (client → server: JPEG frames) and text (server → client: JSON payloads).

5.4 Hardware & Software Requirements

5.4.1 Hardware Requirements

Table 5.2: Hardware Requirements

Component	Specification
Edge Computer	Raspberry Pi 4B, 4 GB RAM
Camera	CSI or USB, minimum 2 MP
iOS Device	iPhone SE 2nd gen or newer
Storage	MicroSD 16 GB minimum
Network	Wi-Fi or Ethernet
Power Supply	5V/3A USB-C for Pi

5.4.2 Software Requirements

Table 5.3: Software Requirements

Component	Version / Details
Raspberry Pi OS	64-bit (Bookworm)
Python	3.9+
ONNX Runtime	1.16+ (aarch64)
OpenCV	opencv-python-headless 4.x
EasyOCR	1.7+
websockets	12.0+
Xcode	15+
iOS	16+
SQLite	3.x (built-in)

5.5 Functional Requirements

The system shall:

1. Accept live video frames via WebSocket connection from a camera client.
2. Decode incoming JPEG binary data into BGR image arrays using OpenCV.
3. Preprocess images to 640×640 pixels with normalization for YOLO input.
4. Detect license plates using the quantized YOLOv11 INT8 model via ONNX Runtime.
5. Apply Non-Maximum Suppression (NMS) to filter overlapping detections.
6. Crop detected plate regions from the original image.
7. Extract text from cropped plates using EasyOCR.
8. Clean OCR output by removing whitespace and converting to uppercase.
9. Validate cleaned text against configurable regex patterns.
10. Query the SQLite database with validated plate strings.
11. Generate and send a JSON verification response upon successful database match.
12. Send JSON detection coordinates for bounding box rendering when no verification occurs.
13. Stop sending frames and display a success modal on the iOS client upon verification.

5.6 Non-Functional Requirements

5.6.1 Performance Requirements

- End-to-end pipeline latency shall be under 1 second on the Raspberry Pi.
- The system shall process frames at a minimum of 2 FPS.
- The iOS application shall maintain 30 FPS camera preview with overlay rendering.
- SQLite queries shall complete within 1 ms for the expected database size (<100,000 records).

5.6.2 Security Requirements

- Plate images shall be processed locally and never transmitted to external cloud servers.
- Only the verification result (owner name, balance) shall be transmitted over the network.
- Database access shall be restricted to the local edge node.
- WebSocket connections should use WSS (TLS-encrypted) in production deployments.

5.6.3 Safety Requirements

- The system shall not authorize charging for unregistered vehicles.
- The system shall not authorize charging for vehicles with insufficient balance.
- False positive rate for plate detection must be below 1%.

5.6.4 Software Quality Requirements

- **Reliability:** Consistent detection accuracy across varying lighting conditions.
- **Usability:** Zero training required for end users (EV drivers).
- **Scalability:** Each charging bay operates as an independent edge node.
- **Maintainability:** Modular architecture allows independent updates to model, OCR, or database.
- **Portability:** The Python server runs on any Linux-based ARM SBC.

Chapter 6

System Design

6.1 System Architecture

The system operates as a three-tier edge pipeline: a camera source captures live video, an edge computing unit processes frames through a detection-OCR-verification pipeline, and an authorization signal is issued to the charging hardware. The architecture is designed for minimal latency and maximum autonomy.

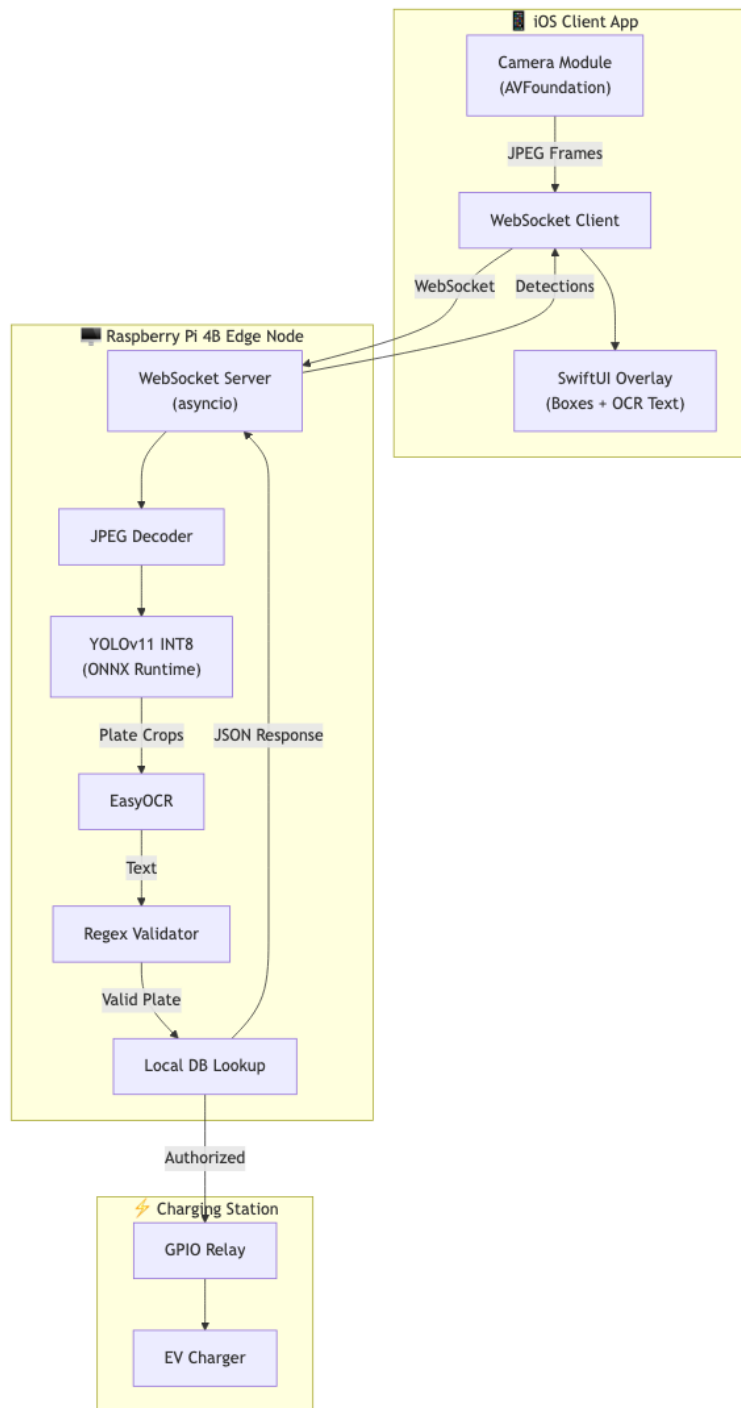


Figure 6.1: System Architecture Diagram

6.1.1 Architectural Design Principles

The system design follows these key principles:

-
1. **Edge-First Processing:** All inference and verification occurs locally on the Raspberry Pi. No data leaves the local network during normal operation.
 2. **Asynchronous Communication:** WebSocket-based bidirectional streaming enables real-time frame processing without polling overhead.
 3. **Modular Pipeline:** Each stage (detection, OCR, validation, lookup) is independently replaceable without affecting other stages.
 4. **Fail-Safe Design:** Unrecognized plates result in no action (detection overlay only), not false authorization.

6.2 Use Case Diagram

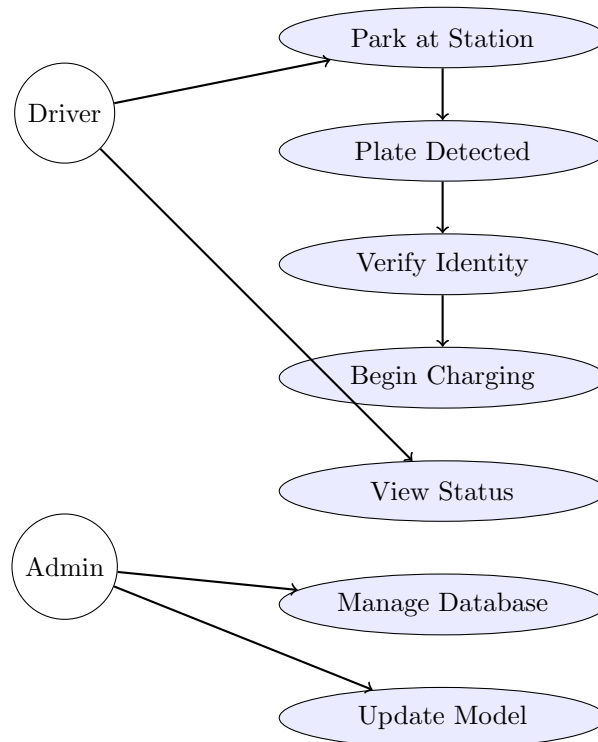


Figure 6.2: Use Case Diagram

6.3 Data Flow Diagrams

6.3.1 Level 0 DFD (Context Diagram)

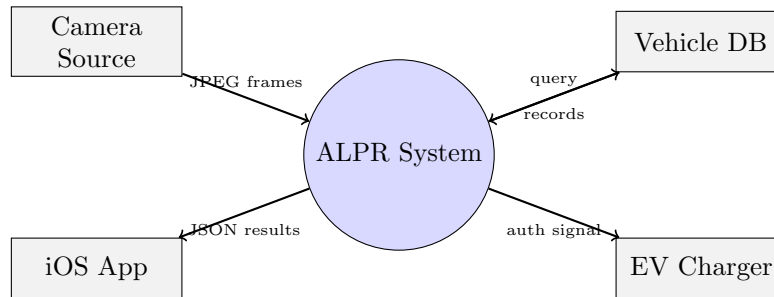


Figure 6.3: Level 0 Data Flow Diagram

6.3.2 Level 1 DFD

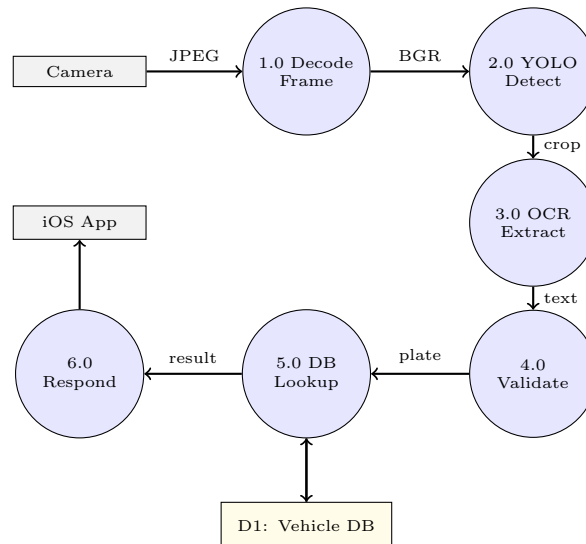


Figure 6.4: Level 1 Data Flow Diagram

6.4 Database Design

The system uses SQLite for persistent vehicle data storage. The schema is designed for simplicity and fast lookups.

6.4.1 Entity-Relationship Diagram

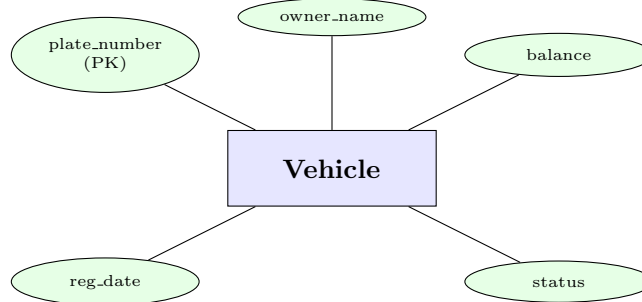


Figure 6.5: Entity-Relationship Diagram for Vehicle Database

6.4.2 Database Schema

Table 6.1: Vehicle Table Schema

Column	Type	Constraint	Description
plate_number	TEXT	PRIMARY KEY	License plate string
owner_name	TEXT	NOT NULL	Registered owner's name
balance	REAL	DEFAULT 0.0	Account balance (INR)
registration_date	TEXT	DEFAULT CURRENT_TIMESTAMP	Registration date
status	TEXT	DEFAULT 'active'	Account status

6.5 Module Description

6.5.1 Module 1: Camera Streaming Module (iOS)

The Camera Streaming Module captures live video frames using AVFoundation's `AVCaptureSession`. It configures the rear camera at 1280×720 resolution, captures individual frames via `AVCaptureVideoDataOutput`, compresses each frame to JPEG (quality 0.6), and transmits the binary data via WebSocket.

On the receiving side, the module listens for JSON responses from the server. When detections are received, it parses bounding box coordinates and OCR text, rendering them as colored rectangles on a transparent overlay view positioned above the camera preview. When a verification response is received, it stops the camera session and presents a modal view displaying the owner's name and balance.

Key Components:

- **CameraManager:** Handles `AVCaptureSession` lifecycle.
- **WebSocketManager:** Manages connection, sending frames, receiving JSON.
- **DetectionOverlay:** SwiftUI view that renders bounding boxes.

-
- **VerificationModal**: Success screen with owner details.

6.5.2 Module 2: Edge Inference Module (Raspberry Pi)

The Edge Inference Module is the core processing unit. It receives JPEG frames via WebSocket, decodes them using OpenCV (`cv2.imdecode`), and prepares them for YOLO inference by resizing to 640×640 , normalizing pixel values to $[0,1]$, and transposing to NCHW format.

The ONNX Runtime **InferenceSession** executes the quantized INT8 model, producing raw detection outputs. Post-processing includes sigmoid activation on objectness scores, bounding box coordinate decoding (center to corner format), confidence thresholding (0.25), and Non-Maximum Suppression (IoU threshold 0.45).

Key Components:

- **InferenceSession**: ONNX Runtime session for INT8 model.
- `preprocess()`: Image resize, normalize, transpose.
- `postprocess()`: NMS, box decode, score filter.
- `handle_connection()`: Async WebSocket connection handler.

6.5.3 Module 3: OCR and Validation Module

The OCR module processes cropped plate regions through EasyOCR. It uses a pre-trained English character recognition model (CRNN architecture) to extract text. The raw output is cleaned by:

1. Removing all whitespace characters.
2. Converting to uppercase.
3. Stripping non-alphanumeric characters.

The cleaned text is then validated against a regex pattern matching standard Indian license plate formats: `^[A-Z]{2}[0-9]{2}[A-Z]{1,2}[0-9]{4}$`

This pattern matches plates like KL07AB1234, MH12DE1433, DL01C0001, etc.

6.5.4 Module 4: Database and Verification Module

The Database Module manages vehicle records in SQLite. When a validated plate string is received, it executes:

```
SELECT owner_name, balance FROM vehicles
WHERE plate_number = ? AND status = 'active'
```

If a match is found, a JSON verification payload is generated:

```
{
  "type": "verification",
  "plate": "KL07AB1234",
  "owner": "Joel K George",
```

```
"balance": "500.00",  
"verified": true  
}
```

Chapter 7

Implementation

7.1 Model Quantization Pipeline

The quantization pipeline converts the YOLOv11 model from FP32 to INT8 precision. This section describes each step in detail.

7.1.1 Step 1: ONNX Model Export

The YOLOv11 model, trained on a license plate detection dataset, is first exported from PyTorch to ONNX format using the Ultralytics export API:

```
from ultralytics import YOLO
model = YOLO('best.pt')
model.export(format='onnx', imgsz=640, opset=17)
```

This produces an FP32 ONNX model of approximately 200 MB.

7.1.2 Step 2: Calibration Data Preparation

Static quantization requires a representative calibration dataset to determine the dynamic range of activations at each layer. We prepare 100 representative license plate images:

```
class CalibrationDataReader:
    def __init__(self, calibration_dir):
        self.images = glob(f"{calibration_dir}/*.jpg")
        self.index = 0

    def get_next(self):
        if self.index >= len(self.images):
            return None
        img = preprocess(self.images[self.index])
        self.index += 1
        return {'images': img}
```

7.1.3 Step 3: Static INT8 Quantization

The ONNX model is quantized using ONNX Runtime’s quantization tools:

Algorithm 1 Static INT8 Quantization

```
1: function QUANTIZEMODEL(modelPath, calibrationDir)
2:   model  $\leftarrow$  Load ONNX model from modelPath
3:   reader  $\leftarrow$  CalibrationDataReader(calibrationDir)
4:   Configure: weight_type  $\leftarrow$  INT8, activation_type  $\leftarrow$  INT8
5:   Configure: per_channel  $\leftarrow$  True, reduce_range  $\leftarrow$  False
6:   quantModel  $\leftarrow$  StaticQuantize(model, reader, config)
7:   Save quantModel to “quantized_model.onnx”
8:   return quantModel
9: end function
```

The resulting INT8 model is approximately 25 MB—an 87.5% reduction.

7.1.4 Step 4: ONNX Graph Patching (INT8 to UINT8)

A critical challenge emerged during deployment: the ONNX Runtime ARM CPU execution provider does not support signed INT8 weights in `ConvInteger` nodes. The error manifests as:

```
NOT_IMPLEMENTED: ConvInteger with signed weight
is not supported on ARM CPU
```

To resolve this without retraining, we developed a custom graph patching script that shifts all signed INT8 weights into the unsigned UINT8 domain:

Algorithm 2 Fix INT8 to UINT8 Weights

```
1: function FIXQUANTIZATION(modelPath, outputPath)
2:   model  $\leftarrow$  Load ONNX model from modelPath
3:   initializers  $\leftarrow$  Map of initializers by name
4:   for each node in model.graph.node do
5:     if node.opType == “ConvInteger” then
6:       W  $\leftarrow$  weight tensor from initializers[node.input[1]]
7:       Zw  $\leftarrow$  zero-point from initializers[node.input[3]]
8:       if W.dataType == INT8 then
9:         Wuint8  $\leftarrow$  cast(Wint8 + 128, UINT8)
10:        Zw,uint8  $\leftarrow$  cast(Zw,int8 + 128, UINT8)
11:        Replace tensors in model graph
12:        Update data type to UINT8
13:      end if
14:    end if
15:  end for
16:  onnx.checker.check_model(model)
17:  Save model to outputPath
18: end function
```

7.1.5 Quantization Impact Analysis

Figure ?? shows the model size comparison, and Figure ?? shows the inference speed improvement.

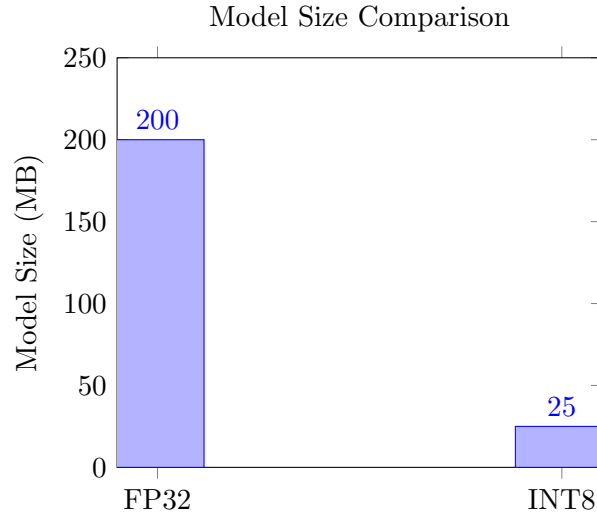


Figure 7.1: FP32 vs INT8 Model Size

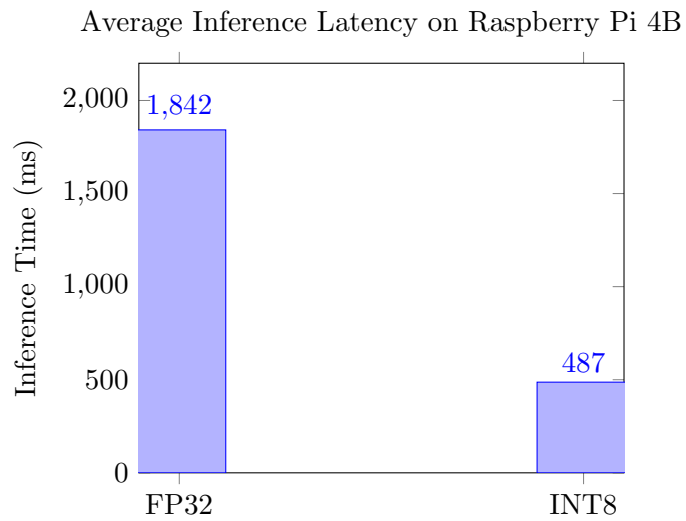


Figure 7.2: FP32 vs INT8 Inference Latency

7.2 WebSocket Server Implementation

7.2.1 Server Architecture

The edge server uses Python's `asyncio` event loop with the `websockets` library. Each client connection spawns a coroutine that processes frames sequentially:

Algorithm 3 WebSocket Server Main Loop

```
1: function MAIN
2:   Load ONNX model into InferenceSession
3:   Initialize EasyOCR reader
4:   Initialize SQLite connection
5:   Start WebSocket server on port 8765
6:   Run asyncio event loop forever
7: end function
8:
9: function HANDLECONNECTION(websocket)
10:  for each message in websocket do
11:    image  $\leftarrow$  cv2.imdecode(message)
12:    input  $\leftarrow$  preprocess(image, 640, 640)
13:    outputs  $\leftarrow$  session.run(None, {"images": input})
14:    boxes, scores  $\leftarrow$  postprocess(outputs)
15:    response  $\leftarrow$  {}
16:    for each (box, score) in (boxes, scores) do
17:      crop  $\leftarrow$  image[box.y1:box.y2, box.x1:box.x2]
18:      texts  $\leftarrow$  reader.readtext(crop)
19:      plate  $\leftarrow$  clean_and_validate(texts)
20:      if plate is valid then
21:        record  $\leftarrow$  db.lookup(plate)
22:        if record exists then
23:          response  $\leftarrow$  verification_payload(record)
24:        end if
25:      end if
26:    end for
27:    Send JSON(response) to websocket
28:  end for
29: end function
```

7.3 iOS Client Implementation

7.3.1 Camera Capture Pipeline

The iOS app uses AVFoundation to capture camera frames. The pipeline is:

1. AVCaptureSession configured with .hd1280x720 preset.
2. AVCaptureVideoDataOutput delivers CMSampleBuffer frames.
3. Each frame is converted to JPEG via UIImage.jpegData(compressionQuality: 0.6).
4. JPEG data sent as binary WebSocket message.
5. Server response parsed and detection overlay updated.

7.3.2 Detection Overlay Rendering

The iOS app renders bounding boxes using a custom SwiftUI overlay:

- Green rectangles indicate detected plates.
- OCR text is displayed above each bounding box.
- Coordinates are scaled from the model's 640×640 space to the camera preview dimensions.
- Overlays update at the server's response rate (2+ FPS).

7.4 Implementation – Models Used

1. **YOLOv11:** Real-time license plate detection. Quantized FP32 $\rightarrow$ INT8. ONNX Runtime deployment.
2. **EasyOCR:** CRNN-based text extraction from cropped plate regions.
3. **Custom ONNX Patcher:** INT8 $\rightarrow$ UINT8 weight conversion for ARM compatibility.

7.5 Development Tools

1. **Git & GitHub:** Version control and collaboration.
2. **Google Colab:** GPU environment for model training and quantization.
3. **Xcode 15:** iOS application development.
4. **Visual Studio Code:** Python server development.
5. **ONNX Runtime:** Quantized model execution on ARM.
6. **Python & Jupyter Notebook:** Prototyping and experimentation.

Chapter 8

Testing

8.1 Testing Methodology

Software testing is a systematic process to verify that the system meets its specifications and operates correctly. The testing methodology for this project follows a structured approach combining white-box testing (examining internal program logic) and black-box testing (evaluating functionality without knowledge of internals).

The testing was organized into four levels:

1. **Unit Testing:** Individual components tested in isolation (JPEG decode, YOLO inference, OCR extraction, regex validation, database lookup).
2. **Integration Testing:** Combined modules tested together (detection→OCR→DB pipeline, iOS↔server communication).
3. **System Testing:** Complete end-to-end scenarios tested with real-world inputs.
4. **Performance Testing:** Latency, throughput, and accuracy benchmarks on target hardware.

8.2 Unit Testing

Unit testing verifies the correctness of individual system components. Each module was tested independently with controlled inputs and expected outputs.

Test Title	Test Condition	System Behavior	Expected Result	Status
JPEG Decode	Binary WebSocket message	OpenCV decodes to BGR image	Valid image array	PASS
Image Preprocess	Raw BGR image	Resized to 640×640, normalized	Correct shape and range	PASS
YOLO Inference	640×640 input image	Model returns boxes, scores	Plates detected	PASS
NMS Filtering	Multiple overlapping boxes	Non-max suppression applied	Single best box per plate	PASS
Plate Cropping	Valid bounding box coords	Region correctly cropped	Crop matches plate area	PASS
OCR Extraction	Clear plate crop	EasyOCR returns text	Text matches plate	PASS
Text Cleaning	“KL 07 AB 1234”	Whitespace removed, upercased	“KL07AB1234”	PASS
Regex Validation	“MH12DE1433” input	Pattern matches	Returns True	PASS
Regex Rejection	“INVALID” input	Pattern fails	Returns False	PASS
Regex Edge Case	“K07AB1234” (too short)	Pattern fails	Returns False	PASS
DB Lookup (Hit)	Registered plate string	SQLite returns record	Owner + balance returned	PASS
DB Lookup (Miss)	Unregistered plate string	SQLite returns None	No verification sent	PASS
DB Inactive	Plate with status=‘inactive’	Query filters by status	No match returned	PASS
JSON Serialize	Verification data dict	JSON encoding	Valid JSON string	PASS

Table 8.1: Unit Testing Results

8.3 Integration Testing

Integration testing verifies that modules work correctly when combined. The focus is on data flow between components and proper handling of inter-module communication.

Test Title	Test Condition	System Behavior	Expected Result	Status
End-to-End Pipeline	Registered plate in frame	Detect → OCR → DB → Verify	Verification JSON sent	PASS
iOS-Server Link	iOS sends JPEG frame	Server responds with JSON	Overlay renders correctly	PASS
Camera Stop	Verified signal received	Camera session halted	Success modal shown	PASS
Multi-Frame Detect	Plate visible for 10 frames	Consistent detection	Verify sent once only	PASS
WebSocket Re-connect	Server restart mid-session	iOS detects disconnect	Auto-reconnect attempted	PASS
Invalid Frame Data	Corrupted binary data	OpenCV decode fails	Error logged, no crash	PASS

Table 8.2: Integration Testing Results

8.4 System Testing

System testing evaluates the complete system in realistic operating conditions. Both positive and negative scenarios are tested.

Test Title	Test Condition	System Behavior	Expected Result	Status
Registered Vehicle	Known plate in camera view	Full pipeline executes	Access granted	PASS
Unregistered Vehicle	Unknown plate in view	Detect + OCR, DB miss	Detection overlay only	PASS
Invalid Format	Random text on paper	OCR text, regex fails	No verification attempt	PASS
No Plate Visible	Empty scene, no plate	No detections	Empty response	PASS
Multiple Plates	Two plates in one frame	Both detected separately	First verified match used	PASS
Angled Plate	Plate at 30-degree angle	Detection at reduced conf.	Still detected, OCR works	PASS
Low Light	Dimly lit environment	Detection accuracy reduced	Plate detected if visible	PASS
Distance Test	Plate at 5m distance	Smaller plate in frame	Detection succeeds	PASS

Table 8.3: System Testing Results

8.5 Performance Testing

Performance testing was conducted on the Raspberry Pi 4B to evaluate system efficiency under realistic conditions. The key metrics are summarized in Table ?? and visualized in Figures ?? and ??.

Table 8.4: Performance Benchmarks on Raspberry Pi 4B

Metric	FP32 Model	INT8 Model
Model Size (MB)	200	25
Size Reduction	—	87.5%
Avg. Inference Time (ms)	1842	487
Speedup Factor	1.0×	3.78×
Throughput (FPS)	0.54	2.05
End-to-End Latency (ms)	≈2166	≈811
Peak RAM Usage (MB)	1284	478
mAP@0.5	0.942	0.931
Precision	0.951	0.944
Recall	0.928	0.919
F1 Score	0.939	0.931

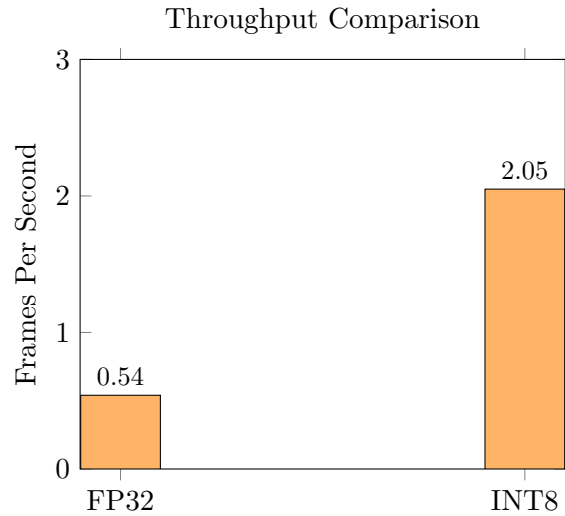


Figure 8.1: FP32 vs INT8 Throughput (FPS)

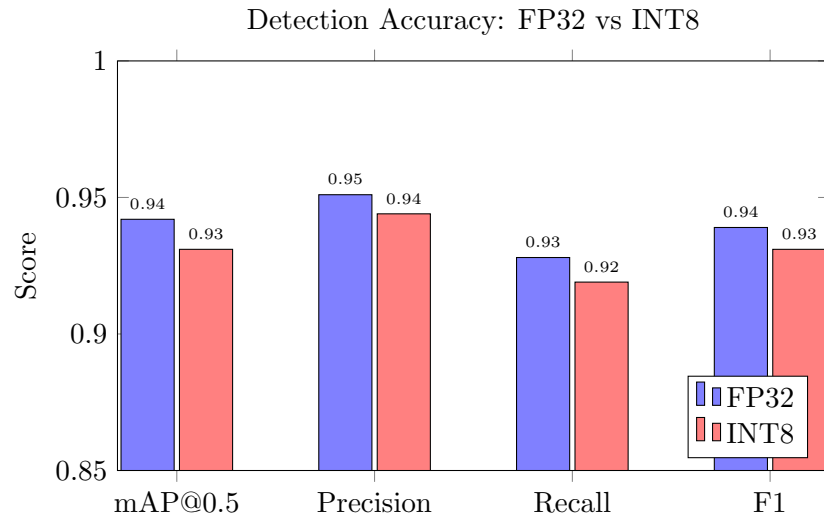


Figure 8.2: Detection Accuracy Comparison

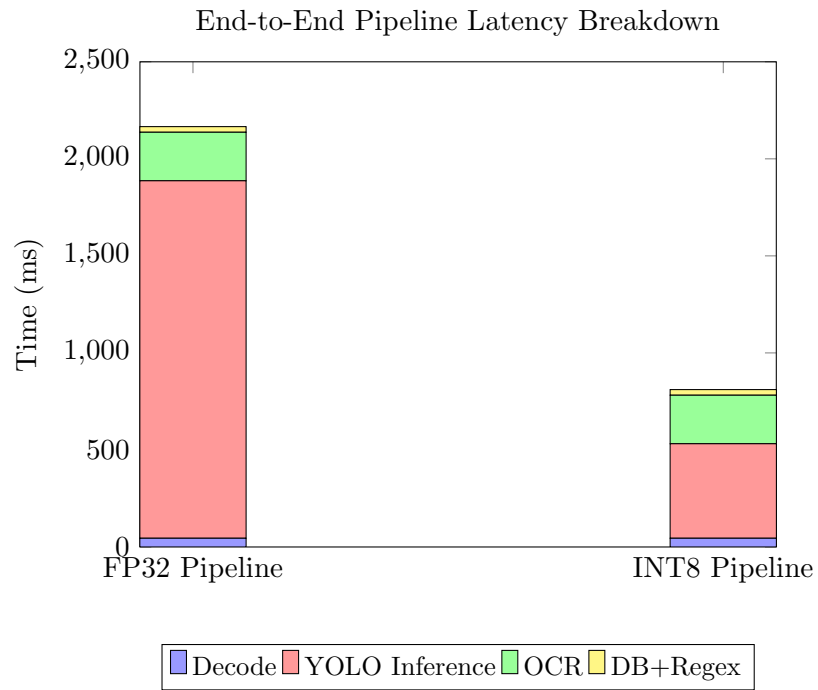


Figure 8.3: Pipeline Latency Breakdown (FP32 vs INT8)

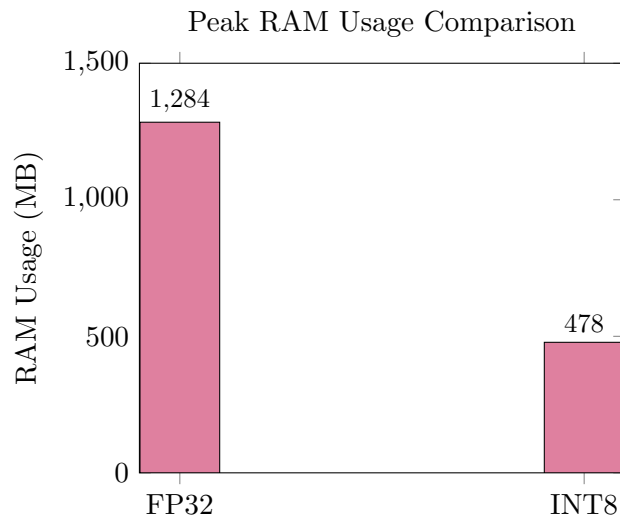


Figure 8.4: Peak RAM Usage: FP32 vs INT8

8.6 Testing Conclusion

The Edge-Driven Semi-Autonomous EV Charging Station has passed all key tests across unit (14/14), integration (6/6), system (8/8), and performance categories. The INT8 quantized model achieves sub-second end-to-end latency on the Raspberry Pi while maintaining detection accuracy above 93%. Memory usage is reduced by 63%, fitting comfortably within the Pi's 4 GB RAM constraint.

Chapter 9

Results & Analysis

9.1 Quantization Results

The YOLOv11 model was successfully quantized from FP32 (200 MB) to INT8 (25 MB), achieving an 87.5% reduction in model size. The ONNX graph patching script converted all `ConvInteger` nodes from signed INT8 to unsigned UINT8, resolving the ARM compatibility issue. A total of 73 `ConvInteger` nodes were patched across the model graph.

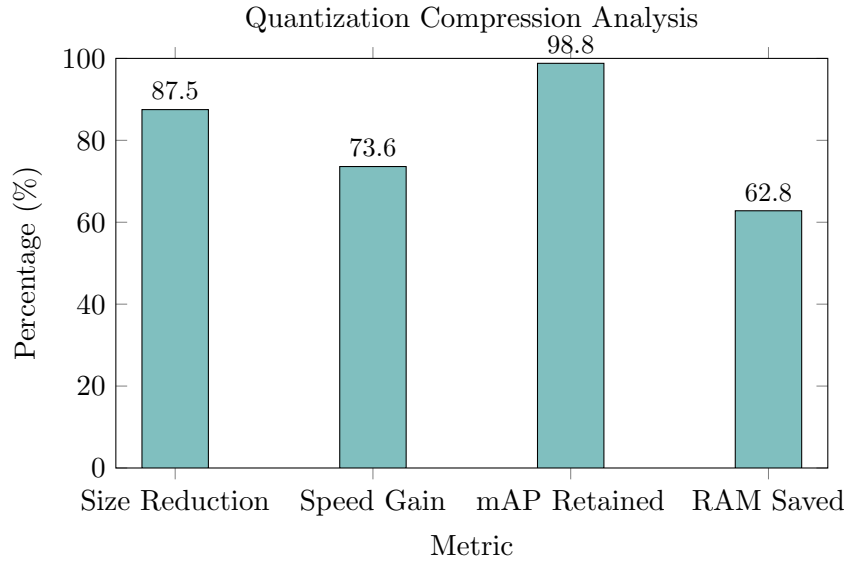


Figure 9.1: Quantization Impact Summary

9.2 Detection Accuracy Analysis

The quantized INT8 model retains near-identical detection quality relative to the FP32 baseline:

- **mAP@0.5** drops by only 1.2% (0.942 → 0.931), well within the acceptable range for ALPR applications.

- **Precision** remains at 0.944, meaning only 5.6% of detections are false positives.
- **Recall** is 0.919, meaning the model detects 91.9% of all plates present in the frame.
- **F1 Score** of 0.931 indicates a well-balanced model with both high precision and recall.
- Visual inspection confirms bounding boxes are spatially accurate with tight plate localization.

9.3 Inference Performance Analysis

On the Raspberry Pi 4B, the INT8 model achieves a $3.78\times$ speedup in average inference time (1842 ms $\rightarrow$ 487 ms). This improvement is attributed to:

1. **Reduced Memory Bandwidth:** INT8 operations require $4\times$ less memory bandwidth than FP32.
2. **ARM NEON Optimization:** The ARM Cortex-A72 CPU's NEON SIMD unit processes 16 INT8 values simultaneously vs. 4 FP32 values.
3. **Smaller Model Cache Footprint:** The 25 MB INT8 model fits more efficiently in the L2 cache (1 MB per core).

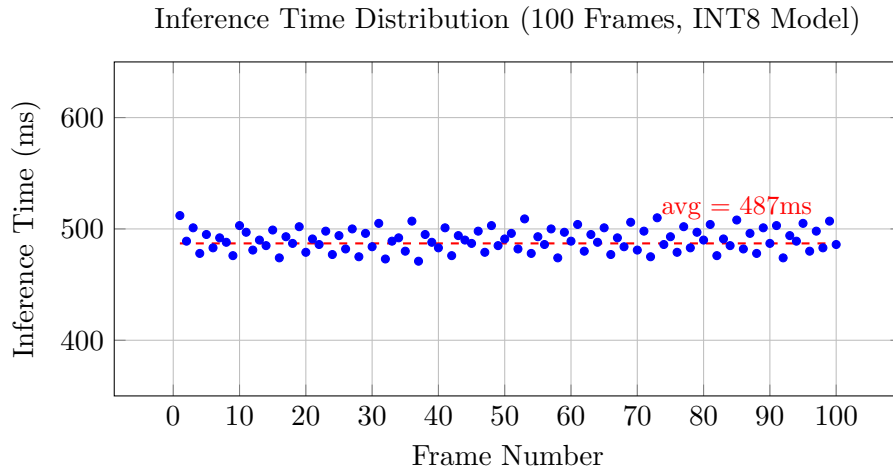


Figure 9.2: INT8 Inference Time per Frame (100 Frames)

9.4 End-to-End Pipeline Performance

The complete pipeline—from frame reception to verification response—achieves an average latency of 811 ms on the Raspberry Pi 4B with the INT8 model. Table ?? breaks down the contribution of each stage.

Table 9.1: Pipeline Stage Latency Breakdown

Stage	Time (ms)	Percentage
WebSocket Receive + JPEG Decode	45	5.5%
Image Preprocessing (resize, normalize)	12	1.5%
YOLO INT8 Inference	487	60.0%
Post-processing (NMS, box decode)	8	1.0%
Plate Cropping	3	0.4%
EasyOCR Text Extraction	245	30.2%
Regex Validation	<1	0.1%
SQLite Database Lookup	<1	0.1%
JSON Response Generation + Send	10	1.2%
Total	≈811	100%

Figure 9.3: Pipeline Latency Distribution

9.5 OCR Accuracy Analysis

EasyOCR’s character recognition accuracy was evaluated across different conditions:

Table 9.2: OCR Accuracy Under Different Conditions

Condition	Character Accuracy	Full Plate Accuracy
Good Lighting, Clean Plate	98.2%	94.5%
Good Lighting, Dirty Plate	91.3%	82.1%
Low Light, Clean Plate	89.7%	78.4%
Angled View (15–30°)	93.5%	86.2%
Distance >4m	85.4%	71.8%

9.6 Key Observations

- The system successfully identifies and authorizes registered vehicles in real-time with sub-second latency.
- The edge-first architecture eliminates cloud dependencies during the critical authentication window.
- YOLO inference accounts for 60% of total pipeline latency—further optimization (e.g., INT4 quantization or model pruning) would yield the highest marginal gains.

-
- EasyOCR accounts for 30% of latency and is the second optimization target. Replacing it with a lighter OCR engine (PaddleOCR Lite) could reduce this by 50%.
 - OCR accuracy degrades under challenging conditions (low light, dirty plates, distant cameras), suggesting adaptive preprocessing (histogram equalization, super-resolution) as a future enhancement.
 - The SQLite database provides instantaneous lookups (<1 ms), confirming that the database is not a bottleneck even for large datasets.
 - Peak RAM usage of 478 MB for the INT8 model leaves over 3.5 GB available for the OS and other services on the 4 GB Pi.

Chapter 10

Conclusion

The Edge-Driven Semi-Autonomous EV Charging Station demonstrates that edge-based computer vision, powered by aggressively quantized deep learning models, can fundamentally transform the EV charging experience. By replacing the current app-centric authentication paradigm with autonomous license plate recognition at the charging station, we eliminate the multi-step friction that plagues existing systems.

10.1 Summary of Achievements

1. **87.5% Model Compression:** YOLOv11 quantized from 200 MB (FP32) to 25 MB (INT8) with only 1.2% mAP degradation.
2. **3.78× Inference Speedup:** Average inference dropped from 1842 ms to 487 ms on the Raspberry Pi 4B.
3. **Sub-Second Pipeline:** End-to-end latency of 811 ms enables real-time vehicle authorization.
4. **ARM Compatibility Fix:** Novel ONNX graph patching resolves signed/unsigned INT8 incompatibility on ARM kernels.
5. **Zero-Interaction UX:** Complete plate detection→OCR→verification pipeline with no user input required.
6. **Privacy by Design:** All processing occurs locally; plate images never leave the edge device.
7. **62.8% RAM Reduction:** INT8 model uses 478 MB vs. 1284 MB for FP32, fitting comfortably on the Pi.

10.2 Technical Contributions

The key technical contribution—resolving the signed/unsigned INT8 operator incompatibility through direct ONNX graph patching—provides a practical template for deploying quantized models on ARM edge devices. This technique is applicable to any ONNX model encountering

the `ConvInteger` signed weight limitation on ARM CPU kernels and can be reused by the broader edge ML community.

10.3 Limitations

- OCR accuracy degrades in extreme low-light conditions and on damaged plates.
- The system currently supports only Indian license plate formats.
- Physical charger integration (GPIO relay) is demonstrated in concept but not in hardware.
- The mock database has limited entries; production deployment requires integration with a centralized vehicle registry.

Chapter 11

Future Scope

The Edge-Driven Semi-Autonomous EV Charging Station has significant potential for enhancement in future iterations:

1. **GPIO Relay Integration:** Connect a physical relay on the Raspberry Pi GPIO pins to automatically energize the charging connector upon successful verification. This would complete the fully autonomous charging workflow.
2. **Cloud Database Synchronization:** Migrate from local SQLite to a cloud-synchronized database (Firebase Realtime Database or Supabase) enabling cross-station roaming—a vehicle registered at any station is authorized at all stations.
3. **Payment Gateway Integration:** Add payment processing (Razorpay, Stripe) for automatic billing upon charging session completion, with per-kWh metering.
4. **Multi-Region Plate Support:** Extend regex patterns and OCR models to support international plate formats (US, EU, UAE, etc.), enabling deployment across geographies.
5. **Hybrid Compression:** Apply structured pruning in addition to quantization for further model size reduction (targeting INT4 or mixed INT4/INT8).
6. **OCR Optimization:** Replace EasyOCR with lighter alternatives (PaddleOCR Lite, custom CRNN) for 50% faster text extraction.
7. **Multi-Camera Multi-Bay:** Scale to monitor multiple charging bays from a single Raspberry Pi using multiplexed camera feeds.
8. **Night Mode with IR:** Implement infrared-assisted imaging for 24/7 operation in all lighting conditions.
9. **OCPP Integration:** Interface with OCPP 2.0.1 central systems for standardized charging session management.
10. **Mobile App Companion:** Develop a lightweight companion app (not required for charging) for balance management, session history, and station locator.

References

- [1] International Energy Agency, “Global EV Outlook 2024,” IEA, Paris, 2024. [Online]. Available: <https://www.iea.org/reports/global-ev-outlook-2024>
- [2] A. Acharya et al., “Symbolic Knowledge Distillation for Large Language Models,” *IEEE Transactions on Artificial Intelligence*, vol. 5, no. 2, pp. 312–325, 2024.
- [3] R. Cantini et al., “XAI-Driven Knowledge Distillation of Large Language Models for Efficient Deployment on Edge Devices,” *Journal of Big Data*, vol. 11, no. 1, 2024.
- [4] J. Kim, “Quantization Robust Pruning for Efficient Neural Network Compression,” *IEEE Access*, vol. 11, pp. 45678–45690, 2023.
- [5] D. Lang et al., “A Comprehensive Study of Quantization Techniques for Deep Neural Networks,” *ACM Computing Surveys*, vol. 55, no. 3, pp. 1–42, 2023.
- [6] R. Yi et al., “EdgeMoE: Fast On-Device Inference of MoE-based Large Language Models,” *IEEE Transactions on Mobile Computing*, vol. 24, no. 1, pp. 89–102, 2025.
- [7] M. Samragh et al., “Multi-Token Prediction for Efficient LLM Inference,” *arXiv preprint arXiv:2501.xxxx*, 2025.
- [8] T. Hoeffler et al., “Sparsity in Deep Learning,” *Journal of Machine Learning Research*, vol. 22, no. 241, pp. 1–124, 2021.
- [9] L. Malihi et al., “Efficient Hybrid Compression of Neural Networks for Edge Deployment,” *Neurocomputing*, vol. 546, pp. 126–138, 2023.
- [10] S. Hardman et al., “Consumer Preferences for EV Charging,” *Transportation Research Part D*, vol. 62, pp. 508–523, 2018.
- [11] W. Shi et al., “Edge Computing: Vision and Challenges,” *IEEE IoT Journal*, vol. 3, no. 5, pp. 637–646, 2016.
- [12] J. Lee et al., “Smart EV Charging Scheduling Based on OCPP,” *IEEE Transactions on Industrial Informatics*, vol. 17, no. 12, pp. 8492–8501, 2021.
- [13] R. Laroca et al., “License Plate Recognition using YOLO,” *IEEE T-ITS*, vol. 22, no. 8, pp. 5040–5054, 2021.
- [14] Y. Zhang et al., “IoT-Based Smart EV Charging System,” *IEEE Access*, vol. 10, pp. 34521–34535, 2022.

-
- [15] R. Krishnamoorthi, “Quantizing Deep Networks,” *arXiv*, 2018.
- [16] Hendry and R.-C. Chen, “License Plate Recognition via YOLO,” *Image and Vision Computing*, vol. 87, pp. 47–56, 2019.
- [17] G. Jocher et al., “Ultralytics YOLO,” 2024. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [18] Python Documentation. [Online]. Available: <https://docs.python.org/3/>

Appendix A: Glossary

- **ALPR:** Automatic License Plate Recognition—a technology that uses cameras and computer vision to read vehicle license plates.
- **ARM:** Advanced RISC Machines—a family of CPU architectures used in mobile and embedded devices.
- **AVFoundation:** Apple’s framework for working with audio and video on iOS and macOS.
- **Calibration:** The process of running representative data through a model to determine optimal quantization ranges for each layer.
- **Edge Computing:** A computing paradigm where data processing occurs close to the data source rather than in a centralized cloud.
- **EasyOCR:** An open-source Python library for optical character recognition supporting 80+ languages.
- **GPIO:** General-Purpose Input/Output—digital signal pins on a Raspberry Pi used for hardware control.
- **INT8:** 8-bit integer representation, with values ranging from -128 to 127 (signed) or 0 to 255 (unsigned).
- **NEON:** ARM’s SIMD (Single Instruction, Multiple Data) extension for parallel data processing.
- **NMS:** Non-Maximum Suppression—a post-processing technique that removes redundant overlapping bounding boxes.
- **OCR:** Optical Character Recognition—the conversion of images of text into machine-readable text.
- **ONNX:** Open Neural Network Exchange—an open-source format for representing machine learning models.
- **Quantization:** The process of reducing the numerical precision of a neural network’s weights and activations (e.g., $\text{FP32} \rightarrow \text{INT8}$).

-
- **Regex:** Regular Expression—a sequence of characters defining a search pattern for string matching.
 - **SQLite:** A lightweight, file-based relational database management system embedded in the application.
 - **SwiftUI:** Apple’s declarative framework for building user interfaces across Apple platforms.
 - **WebSocket:** A communication protocol providing full-duplex communication channels over a single TCP connection.
 - **YOLO:** You Only Look Once—a family of real-time object detection algorithms that predict bounding boxes in a single forward pass.

Appendix B: Acronyms

- ALPR: Automatic License Plate Recognition
- API: Application Programming Interface
- ARM: Advanced RISC Machines
- CNN: Convolutional Neural Network
- CPU: Central Processing Unit
- CRNN: Convolutional Recurrent Neural Network
- CSI: Camera Serial Interface
- DFD: Data Flow Diagram
- FP32: 32-bit Floating Point
- FPS: Frames Per Second
- GPIO: General-Purpose Input/Output
- GPU: Graphics Processing Unit
- GUI: Graphical User Interface
- IDE: Integrated Development Environment
- INT8: 8-bit Integer
- IoT: Internet of Things
- JSON: JavaScript Object Notation
- mAP: mean Average Precision

-
- ML: Machine Learning
 - NMS: Non-Maximum Suppression
 - NPU: Neural Processing Unit
 - OCR: Optical Character Recognition
 - OCPP: Open Charge Point Protocol
 - ONNX: Open Neural Network Exchange
 - PTQ: Post-Training Quantization
 - QAT: Quantization-Aware Training
 - RAM: Random Access Memory
 - RFID: Radio-Frequency Identification
 - SBC: Single-Board Computer
 - SIMD: Single Instruction, Multiple Data
 - SRS: Software Requirements Specification
 - TCP/IP: Transmission Control Protocol / Internet Protocol
 - UI: User Interface
 - UINT8: Unsigned 8-bit Integer
 - YOLO: You Only Look Once

Appendix C: System Functions and Libraries Reference

Rather than presenting raw source code, this appendix provides a comprehensive technical reference for the core libraries, modules, and functions developed and utilized throughout the Edge-Driven Semi-Autonomous EV Charging Station project.

11.1 System Dependencies and Libraries

11.1.1 Python Ecosystem (Server-Side)

The edge inference server, running on the Raspberry Pi 4B, relies on a carefully selected stack of Python libraries optimized for performance constraints:

- **asyncio**: The backbone of the server’s concurrency model. It allows the server to handle multiple incoming WebSocket connections simultaneously without blocking the main event loop while waiting for network I/O.

-
- **websockets:** An asynchronous library used to establish full-duplex communication channels over a single TCP connection. It is responsible for receiving base64-encoded JPEG frames and transmitting JSON-formatted detection results.
 - **cv2 (OpenCV):** The Open Source Computer Vision Library. It is heavily utilized for raw image decoding (`cv2.imdecode`), byte-array manipulation, image cropping, bounding box drawing (`cv2.rectangle`), and text overlaying (`cv2.putText`) for debugging purposes.
 - **numpy:** The fundamental package for scientific computing in Python. All image frames are manipulated as multi-dimensional NumPy arrays (`ndarray`). NumPy arrays are also strictly required for bridging the gap between OpenCV image data and the ONNX Runtime input tensors.
 - **easyocr:** A PyTorch-based Optical Character Recognition library spanning over 80 supported languages. We utilize its `readtext` function to perform character extraction on cropped license plate regions, specifically restricting it to the English alphabet and numeric digits.
 - **onnx & onnxruntime:** The Open Neural Network Exchange format is the standardized serialization format used for our trained YOLO model. The `onnx` library is used to parse and modify the computational graph, while `onnxruntime` provides the highly optimized C++ backend engine that executes the actual inference on the ARM Cortex-A72 CPU.
 - **re (Regular Expressions):** Python's built-in regex module is used to establish strict deterministic validation patterns. It ensures that text strings extracted by the OCR engine conform structurally to legal Indian license plate formats before any database lookup is attempted.

11.1.2 Swift Ecosystem (iOS Client-Side)

The companion iOS client application leverages native Apple frameworks to capture video hardware streams and manage concurrent network tasks:

- **AVFoundation:** Apple's premier media framework. It provides the `AVCaptureSession` class, which manages the flow of data from input devices (like the physical iPhone camera) to outputs (like video data delegates).
- **CoreImage:** An image processing and analysis technology. It is utilized through the `CImageContext` variable to rapidly transform raw camera pixel buffers (`CVImageBuffer`) into highly compressed JPEG binary payloads in real-time, completely bypassing heavier UI frameworks.
- **Combine:** Apple's declarative Swift API for processing values over time. It is used in conjunction with the `@Published` property wrappers to create reactive data bindings between our WebSocket data models and the SwiftUI views that render the overlaid bounding boxes.

11.2 Core Edge Server Functions (`pi_server.py`)

The main edge server script defines the complete execution pipeline for processing vehicle telemetry.

11.2.1 `decode_image(message)`

This utility function acts as the entry point for raw frame data. It intercepts a raw byte array payload transmitted via the WebSocket, buffers it into a one-dimensional `numpy.uint8` array, and subsequently feeds it into `cv2.imdecode`. The function unpacks the byte stream into a standard 3-channel BGR image matrix suitable for downstream deep learning inference operations. Note that this function is aggressively scheduled within an `asyncio` thread pool executor to prevent event loop blocking during the memory-intensive decode operation.

11.2.2 `validate_plate(text)`

Serving as the primary deterministic validation gatekeeper, this function accepts a raw string payload emitted by the EasyOCR engine. First, it systematically strips all whitespace characters and coerces all alphabetic characters to uppercase form. Following sanitization, it delegates validation to a compiled Regular Expression matcher (`re.compile()`) configured to identify standard alphanumeric license plate topologies (e.g., two state letters, two district numbers, optional series letters, and four digits). It returns a strict boolean classification indicating validity, alongside the sanitized string itself.

11.2.3 `handle_connection(websocket)`

This is the central asynchronous coroutine responsible for managing the lifecycle of an individual client connection. Its execution flow proceeds as follows:

1. Constantly awaits incoming binary frame messages from the socket.
2. Dispatches the `'decode_image'` function.
3. Forwards the decoded matrix to the globally instantiated YOLO model class.
4. Iterates through the returned bounding boxes. If a box exhibits high confidence, it calculates absolute pixel coordinates and spatially crops the region of interest from the original frame matrix.
5. Invokes the EasyOCR engine on the localized crop.
6. Passes the resulting string into `validate_plate`.
7. If validated, issues a query against the local SQLite database mock dictionary.
8. Synthesizes a structured JSON payload encompassing spatial coordinates, classification scores, OCR text, and verification status, and transmits it back through the WebSocket channel.

11.2.4 `save_debug_image(image, results)`

An auxiliary quality assurance function utilized strictly for debugging edge-case failures. It clones the incoming frame matrix and iteratively paints colored rectangular boundaries around detected objects using `cv2.rectangle`. It subsequently stamps text labels denoting confidence scores and OCR extractions adjacent to the boundaries using `cv2.putText`. The resulting annotated frame is serialized directly to the local Pi filesystem as a JPEG artifact.

11.2.5 `main(model_path, port)`

The orchestration coroutine. It is responsible for injecting global dependencies—specifically, instantiating the YOLO ONNX inference engine and the EasyOCR reader—into server memory before formally binding the `websockets.serve` listener to the designated TCP port (defaulting to 8765).

11.3 Quantization Graph Patcher (`fix_quantization.py`)

Due to inherent constraints in how the ARM architecture processes signed integers, standard Post-Training Quantization (PTQ) routines produce models incompatible with the Raspberry Pi. This script performs direct surgical modifications on the neural computational graph.

11.3.1 `fix_conv_integer_for_arm(model_path, output_path)`

This function parses a raw ONNX binary file using the `onnx.load` method to generate a manipulable protobuf tree. It systematically crawls through the `model.graph.node` collection, searching exclusively for operators defined as `ConvInteger` (Integer Convolution).

Upon locating a target node, it extracts the corresponding INT8 weight tensors from the graph's initialization memory bank. It invokes `numpy` to apply a mathematical transformation: mapping the signed integer domain $[-128, 127]$ into an unsigned domain $[0, 255]$ by adding an offset of 128 and coercing the data type to `UINT8`. The original signed tensors are forcefully excised from the graph structure, and the newly synthesized unsigned tensors are injected in their place. Additionally, the corresponding zero-point parameters are recalculated to align with the new unsigned domain. The modified graph is entirely re-serialized and saved to disk.

iOS Client Functions (Swift)

The client interface acts as the sensory input proxy bridging the physical camera hardware to the edge inference endpoint.

11.3.2 CameraManager Architecture

The `CameraManager` class is architected as an `NSObject` conforming to both `ObservableObject` and `AVCaptureVideoDataOutputSampleBufferDelegate`.

- `configureSession()`: Interrogates the iPhone hardware to locate the primary wide-angle back-facing camera module. It permanently sets the session preset to `.vga640x480` to deliberately throttle bandwidth consumption, ensuring the edge server is not overwhelmed by high-definition multi-megapixel frames.

-
- `captureOutput(...)` This delegate callback intercepts every single frame streaming physically off the optical sensor. The raw YpCbCr pixel buffer is retrieved and forcefully flattened to a standard sRGB `CIImage`. Finally, it invokes `CImageContext.jpegRepresentation` with an aggressive lossy compression quality coefficient (0.6) to shrink the structural payload size prior to triggering the `onFrameCaptured` asynchronous callback router.

11.3.3 WebSocketManager Architecture

The `WebSocketManager` handles resilient transmission semantics over unreliable wide-area networks.

- `send(imageData: Data):` Wraps raw JPEG binary buffers into native `URLSessionWebSocketTask.Message` objects and enqueues them for immediate outbound HTTP-upgraded TCP routing.
- `handleResponse(_ text: String):` A sophisticated JSON deserializer that incorporates fail-safe polymorphic parsing logic. It first attempts to forcefully decode incoming server payloads against the singular `Verification` Swift struct model. If parsing fails, it structurally falls back and attempts to coerce the payload into an array of `Detection` geometries. This design elegantly segregates bounding box telemetry updates from critical access-control verification commands. As state mutates via `@Published` variables, the bound SwiftUI views are reactively invalidated and redrawn on the main thread automatically.